GOODROAD: CITIZEN-POWERED TRAFFIC VIOLATION REPORTING SYSTEM

Hendawitharana H W K P

IT22219466

BSc (Hons) degree in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

PENALTY ENFORCEMENT AND REWARD DISTRIBUTION USING DETERMINISTIC LOGIC AND GENERATIVE AI

Hendawitharana H W K P

IT22219466

BSc (Hons) degree in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION OF THE CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works.

Name	Student ID	Signature
Hendawitharana H W K P	IT22219466	

The supervisor/s should certify the dissertation with the following declaration.

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

…….. ……..

Signature of the supervisor Date

…….. ……..

Signature of the co-supervisor Date

ABSTRACT

The "Penalty Enforcement and Reward Distribution" module serves as the administrative and financial core of the GoodRoad Citizen-Powered Traffic Violation Reporting System. Current road traffic enforcement systems are either purely manual or insufficiently adaptive, resulting in delayed penalization, weak deterrence for repeat offenders, and the underutilization of citizen-driven reporting. This research introduces a hybrid enforcement component that combines a deterministic legal-point computation engine with automated, context-aware Generative AI notifications, while introducing a transparent monetary distribution model.

The backend architecture, engineered using FastAPI and PostgreSQL via SQLAlchemy, implements a strict deterministic rule set covering 30 distinct traffic violations. The system computes penalty severity using dynamic frequency-based multipliers and assigns temporal expiry windows. Furthermore, it executes a transparent 60/25/15 financial distribution algorithm, directly incentivizing public participation by allocating 25% of validated fines to a citizen reward pool. To transition enforcement from punitive to educational, the Google Gemini API autonomously drafts personalized warning emails, supported by a deterministic fallback template to ensure system reliability. The resulting prototype successfully demonstrates a highly scalable, interpretable, and extensible enforcement ecosystem that modernizes traffic governance.

Keywords: Traffic Enforcement, Deterministic Logic, Generative AI, Revenue Distribution, FastAPI, PostgreSQL.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Dr. Mahima Weerasinghe, for their invaluable guidance, continuous support, and profound technical insights throughout the lifecycle of this research. Their expertise and encouragement were instrumental in shaping the architectural direction of the Penalty Enforcement and Reward Distribution module, and their mentorship was vital to the overall success of the GoodRoad project.

I also extend my sincere appreciation to my dedicated group members – Liyanage R. L. N. L., Rathnayake W. K. D., and Wijerathna P. G. S. P. Our collaborative effort, shared vision, and countless hours of teamwork made the seamless integration of this complex Citizen-Powered Traffic Violation Reporting System possible. Building the bridge between the computer vision modules and the enforcement backend was a significant challenge that we successfully navigated together as a team.

Furthermore, I wish to acknowledge the Department of Information Technology and the Faculty of Computing at the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic foundation, resources, and environment necessary to undertake and complete this dissertation.

Finally, my heartfelt thanks go to my family and friends for their unwavering patience, motivation, and moral support during the most demanding phases of this software development and research journey.

TABLE OF CONTENTS

DECLARATION OF THE CANDIDATE AND SUPERVISOR 3
ABSTRACT 4
ACKNOWLEDGEMENT 5
TABLE OF CONTENTS 6
LIST OF FIGURES 8
LIST OF TABLES 8
LIST OF ABBREVIATIONS 9

1. INTRODUCTION 10
1.1 Background and Literature Survey 10
1.1.1 The Limitations of Traditional Automated Enforcement 11
1.1.2 Artificial Intelligence in Judicial and Financial Adjudication 11
1.1.3 Citizen-Sourced Enforcement and Incentive Models 12
1.1.4 Generative AI in Public Sector Communication 12
1.2 Research Gap 13
1.2.1 The Gap in Legal Determinism vs. Predictive AI 13
1.2.2 The Absence of Programmatic Financial Distribution Models 14
1.2.3 Deficiencies in Context-Aware Driver Communication 15
1.2.4 Inadequate Relational Data Integrity in Civic Platforms 16
1.2.5 Comparative Analysis of Technology Stacks for Legal Adjudication 17
1.3 Research Problem 18
1.4 Research Objectives 18
1.4.1 Main Objective 19
1.4.2 Specific Objectives 19

2. METHODOLOGY 20
2.1 Architectural Overview 21
2.2 Data Model and Relational Integrity 21
2.3 System Dynamics and Sequence Flow 22
2.4 Security Protocols and API Hardening 24

2.5 Deterministic Logic Formulation 25
2.5.1 Repeat-Offense Multiplier Algorithm 25
2.5.2 Financial Splitting Algorithm 26
2.5.3 Profile Stratification Matrices 27
2.6 Generative AI Engine Integration 29
2.7 Commercialization Aspects of the Product 30
2.8 Testing and Implementation 31

3. RESULTS AND DISCUSSION 34
3.1 Frontend Implementation and State Management 34
3.2 Functional Results 37
3.3 Research Findings 39
3.4 Discussion and Identified System Constraints 40
3.5 Summary of Individual Contribution 42

4. CONCLUSIONS 44
REFERENCE LIST 46
APPENDICES 48
APPENDIX A: COMPLETE DETERMINISTIC VIOLATION RULE SET 49
APPENDIX B: POSTGRESQL ORM SCHEMA IMPLEMENTATION 51
APPENDIX C: FASTAPI ROUTING AND CONTROLLER LOGIC 54
APPENDIX D: AI SERVICE AND FALLBACK GENERATION LOGIC 56
APPENDIX E: FRONTEND REACT DASHBOARD IMPLEMENTATION 57
APPENDIX F: POSTMAN API COLLECTION FOR END-TO-END TESTING 57

LIST OF FIGURES

Figure 1.1: Number of road traffic accidents and deaths in Sri Lanka (2013-2021) 2
Figure 1.2: The bottleneck in traditional Automated Traffic Enforcement workflows 13
Figure 2.1: Proposed System Workflow of the Penalty Enforcement Module 18
Figure 2.2: Relational Database Entity Relationship Diagram (ERD) 21
Figure 2.3: UML Sequence Diagram mapping the API request lifecycle 23

LIST OF TABLES

Table 1.1: Comparison of Stochastic AI vs. Deterministic Logic in Legal Adjudication ... 7
Table 2.1: Deterministic Violation Weights and Expiry Matrix 28
Table 2.2: Integration Testing Results for API Endpoints 32

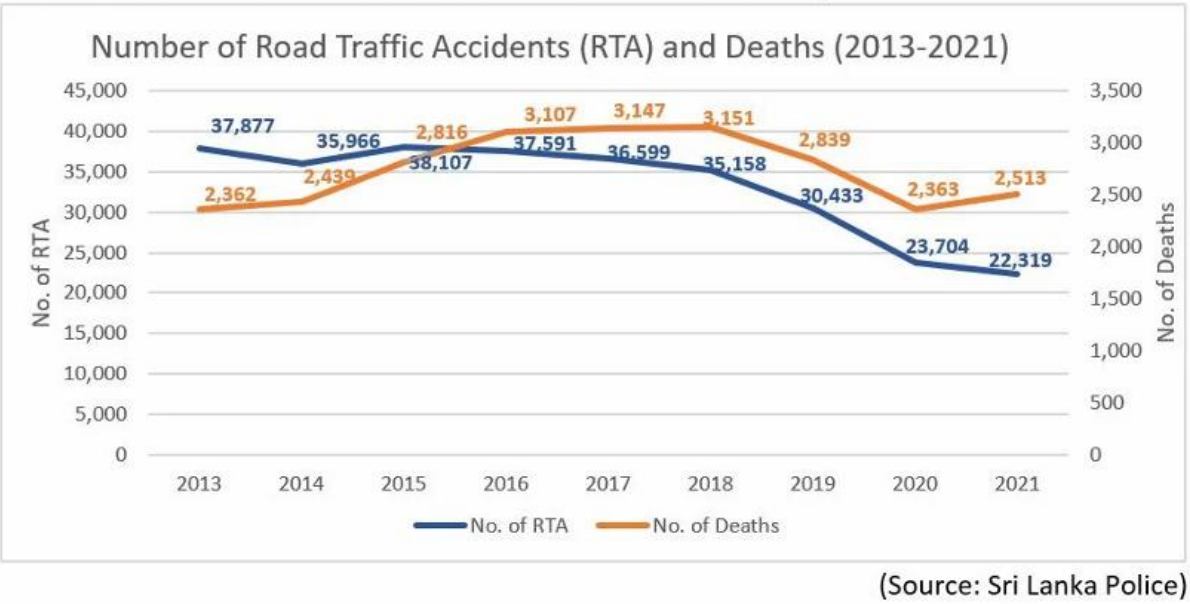
LIST OF ABBREVIATIONS

Abbreviation	Description
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
API	Application Programming Interface
ATES	Automated Traffic Enforcement Systems
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity Relationship Diagram
GenAI	Generative Artificial Intelligence
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LKR	Sri Lankan Rupee
LLM	Large Language Model
LPR	License Plate Recognition
ML	Machine Learning
NLP	Natural Language Processing
ORM	Object-Relational Mapper
RDBMS	Relational Database Management System
REST	Representational State Transfer
RL	Reinforcement Learning
SDK	Software Development Kit
SQL	Structured Query Language
TTL	Time-To-Live
UI	User Interface
UX	User Experience
VLM	Vision-Language Model

1. INTRODUCTION

1.1 Background and Literature Survey

The modernization of urban traffic law enforcement is a critical intersection of public safety, civic administration, and advanced software engineering. Globally, the failure to adequately monitor road networks and enforce traffic laws contributes to approximately 1.3 million annual fatalities and over $518 billion in economic losses. In densely populated developing nations, such as Sri Lanka, traditional enforcement mechanisms are severely constrained by manual resource limitations. With an average ratio of one traffic officer per 450 vehicles, nearly 99% of the road network remains unmonitored at any given time. The staggering human cost of this enforcement gap is illustrated in Figure 1.1 below.



(Source: Sri Lanka Police)

Figure 1.1: Number of road traffic accidents and deaths in Sri Lanka (2013-2021)

While recent decades have seen the introduction of Automated Traffic Enforcement Systems (ATES) – primarily utilizing static speed cameras and closed-circuit television (CCTV)these systems suffer from significant geographical and administrative limitations. To contextualize the development of the Penalty Enforcement and Reward Distribution module, it is necessary to examine the existing literature surrounding traffic enforcement technologies, the application of Artificial Intelligence in legal adjudication, and emerging citizen-participation frameworks.

1.1.1 The Limitations of Traditional Automated Enforcement

Current literature on ATES highlights a fundamental bottleneck: the disconnect between violation detection and administrative penalization. Studies evaluating public technology-based traffic enforcement programs indicate that while cameras can capture raw footage of an infraction, the subsequent adjudication process is highly manual and resource-intensive. When a violation is captured, human operators must typically review the footage, manually determine the appropriate fine based on motor traffic laws, and initiate a delayed postal notification. This delay severs the psychological link between the offense and the consequence, significantly weakening the deterrence effect. The literature demonstrates that for enforcement to effectively rehabilitate driver behavior, the penalty notification must be immediate, transparent, and contextually clear.

1.1.2 Artificial Intelligence in Judicial and Financial Adjudication

The rapid advancement of Machine Learning (ML) has led to exploratory research into fully automating the penalty calculation process. Early theoretical frameworks, including initial proposals for the GoodRoad system, hypothesized the use of Reinforcement Learning (RL) to dynamically calculate fine amounts and penalty severities based on driver behavior profiles. Recent research, such as Chen and Wang's work on dynamic incentive strategies using reinforcement learning, showcases the ability of AI to optimize complex network behaviors.

However, a critical review of software engineering principles in legal contexts reveals a severe vulnerability in this approach. AI models, particularly RL and deep learning neural networks, are stochastic; they operate on probabilistic weights and confidence scores. Traffic law, conversely, demands absolute deterministic consistency. A driver committing a specific offense must receive the exact legal penalty prescribed by the Motor Traffic Act, without the risk of an AI "hallucinating" a higher or lower fine based on opaque training parameters. The literature emphasizes that while AI is highly suited for computer vision tasks (like License Plate Recognition or deepfake detection), the financial and judicial layer must rely on hardcoded, deterministic mathematical logic to maintain legal defensibility and public trust.

1.1.3 Citizen-Sourced Enforcement and Incentive Models

To bridge the 99% unmonitored enforcement gap, global municipalities are increasingly turning to "Citizen-as-Sensor" networks. For example, the "Traffic Sentinel" programs in Indian cities like Delhi and Hyderabad allow citizens to upload photographic evidence of traffic violations, resulting in the issuance of tens of thousands of automated fines. Furthermore, some programs actively incentivize this behavior; reports indicate citizens in Delhi can earn substantial monetary rewards for reporting offenses.

Despite these programmatic successes, the literature reveals a significant architectural gap in the backend software. Existing platforms lack an automated, programmatic financial distribution engine. Rewards are often processed manually via administrative backlogs rather than through immediate algorithmic splits. There is a clear necessity for a robust, mathematically sound distribution model—such as a 60/25/15 revenue split—managed by a strictly relational database (like PostgreSQL) to ensure ACID (Atomicity, Consistency, Isolation, Durability) compliance during financial disbursements.

1.1.4 Generative AI in Public Sector Communication

Historically, traffic fines have been issued as generic, punitive text documents containing legal codes that drivers struggle to understand. While early interactive solutions proposed NLP-powered chatbots to allow drivers to query their violations, user experience research in law enforcement indicates that drivers do not want conversational engagement regarding fines; they require immediate, indisputable evidence.

The advent of Large Language Models (LLMs) and Vision-Language Models offers a novel solution. Current literature on Generative AI in civic administration is shifting toward automated, context-aware "push" notifications. By integrating APIs such as Google Gemini, a system can ingest raw telemetry data (e.g., speed, location, law violated) and autonomously synthesize a personalized, educational warning. Crucially, when paired with the direct attachment of visual photographic evidence within the communication, this approach transitions the enforcement paradigm from merely punitive to actively educational, vastly improving the likelihood of behavioral correction.

1.2 Research Gap

Despite the rapid proliferation of computer vision models—such as YOLO-based architectures for License Plate Recognition (LPR) and Convolutional Neural Networks (CNNs) for vehicle tracking – a profound architectural gap remains in the subsequent administrative and judicial enforcement layers. The transition from detecting a traffic violation to executing a legally sound, financially transparent, and educational penalty remains heavily reliant on outdated manual processes or technologically flawed implementations. An extensive review of current Automated Traffic Enforcement Systems (ATES) and emerging citizen-reporting platforms reveals four critical deficiencies that the GoodRoad Penalty Enforcement and Reward Distribution module seeks to resolve.

1.2.1 The Gap in Legal Determinism vs. Predictive AI

Currently, a significant portion of academic research in intelligent transportation focuses on applying Machine Learning (ML) and Reinforcement Learning (RL) to optimize traffic flows and predict driver behavior. Some theoretical models have attempted to extend these predictive algorithms into the realm of penalty adjudication—utilizing AI to dynamically assess fine amounts based on environmental context or driver history. However, this introduces a severe legal vulnerability. Traffic enforcement fundamentally requires strict determinism; the law dictates precise penalties for specific infractions. Stochastic AI models, which rely on probabilistic weights and confidence thresholds, cannot provide the mathematical guarantee of equal treatment under the law. There is a distinct lack of hybrid enforcement architectures that decouple the stochastic visual detection phase from a strictly deterministic, rule-based legal adjudication engine that can handle repeat-offense multipliers and temporal penalty expirations with absolute mathematical certainty. Table 1.1 outlines the critical differences between these two approaches and justifies the necessity for a rule-based system.

Table 1.1: Comparison of Stochastic AI vs. Deterministic Logic in Legal Adjudication

Feature	Stochastic AI (RL/ML)	Deterministic Logic (Proposed System)
Predictability	Low (Probabilistic)	Absolute 100%
Legal Defensibility	Weak ("Black Box")	Strong (Rule-based)
Output generation	Dynamic floating weights	Fixed legal statutes

1.2.2 The Absence of Programmatic Financial Distribution Models

While global municipalities are increasingly piloting "Citizen-as-Sensor" networks to bridge the massive enforcement monitoring gap, these programs suffer from severe administrative bottlenecks regarding reward distribution. In systems where citizens are financially incentivized to report traffic violations, the backend processing is almost universally manual. Revenue collected from traffic fines is deposited into generalized municipal accounts, and citizen rewards are disbursed through slow, bureaucratic backlogs. A critical gap exists in the software engineering of these systems: the lack of a transparent, programmatic financial splitting algorithm. There is a demonstrated need for a system capable of autonomously mapping computed penalty points to a fiat currency value (e.g., LKR), and executing an algorithmic, atomic split (such as a 60% government, 25% reward, 15% system maintenance distribution) immediately upon fine validation.

1.2.3 Deficiencies in Context-Aware Driver Communication

Standard ATES communicate with traffic violators via automated, generic postal citations or templated SMS notifications. These communications typically cite a legal code and a monetary fine but lack contextual explanation, severely limiting their educational impact. Furthermore, while experimental systems have proposed using Natural Language Processing (NLP) chatbots to allow drivers to query their infractions, this "pull-based" communication requires the driver to initiate the learning process, which rarely occurs. There is a notable absence of systems leveraging advanced Generative AI (GenAI) and Vision-Language Models to proactively synthesize "push-based" educational interventions. A significant opportunity exists to utilize APIs like Google Gemini to ingest raw violation telemetry, automatically draft a personalized explanation of the specific danger posed by the maneuver, and attach verifiable photographic evidence directly into an automated email dispatch.

1.2.4 Inadequate Relational Data Integrity in Civic Platforms

Finally, many rapid prototypes for civic reporting applications utilize NoSQL or document-based databases for speed and flexibility during development. However, the management of legal penalty points, temporal expirations, and financial ledgers requires enterprise-grade data integrity. Document stores are highly susceptible to data anomalies such as orphan records (e.g., retaining a violation record after a driver's profile has been deleted). The literature points

to a gap in deploying robust, normalized relational database schemas—enforced via strict Object-Relational Mappers (ORMs) with cascading referential integrity—specifically tailored for the unique data structures of crowd-sourced traffic enforcement and reward ledgers.

1.2.5 Comparative Analysis of Technology Stacks for Legal Adjudication

The architectural selection for traffic enforcement platforms heavily dictates the legal validity and scalability of the system. A critical gap in existing civic tech prototypes is the misapplication of development frameworks.

- **Backend Frameworks (FastAPI vs. Django/Flask):** Traditional systems utilizing WSGI (Web Server Gateway Interface) synchronous frameworks like Django suffer from blocking I/O operations, which causes severe latency when processing thousands of concurrent dashcam video uploads and API requests. The GoodRoad system addresses this gap by implementing FastAPI, an ASGI (Asynchronous Server Gateway Interface) framework. FastAPI natively supports asynchronous execution (async/await), allowing the server to handle non-blocking concurrent requests while waiting for the Gemini AI API to respond, thereby maximizing CPU utilization and drastically reducing response latency.
- **Database Management (PostgreSQL vs. MongoDB):** Document-based NoSQL databases like MongoDB offer rapid schema iteration, which is advantageous for unstructured data. However, the Penalty Enforcement module fundamentally operates as a financial ledger (managing the 60/25/15 split) and a legal point tracker. NoSQL systems lack robust, native ACID (Atomicity, Consistency, Isolation, Durability) compliance across multi-document transactions. The transition to PostgreSQL, a strict Relational Database Management System (RDBMS), ensures that a financial reward is never disbursed if the corresponding violation validation fails, utilizing atomic commits and rollbacks to guarantee ledger integrity.
- **Vision-Language Models (Google Gemini vs. OpenAI GPT):** In selecting the Generative AI engine, low-latency inference and cost-scalability are paramount. While models like GPT-4 offer high reasoning capabilities, their inference time and token costs are prohibitive for real-time, state-wide traffic enforcement. The Google Gemini 1.5 Flash architecture was strategically selected due to its optimized inference speed

for structured JSON/text generation and multimodal capabilities, allowing the system to synthesize educational warnings in under 2.5 seconds, ensuring the "push-based" notification pipeline remains highly responsive.

1.3 Research Problem

The fundamental problem addressing modern traffic governance is not a lack of detection capability, but rather a profound, systemic deficiency in the subsequent administrative, judicial, and financial execution of enforcement. Over the past decade, immense academic and commercial resources have been poured into the "front-end" of intelligent transportation—specifically, training computer vision models like YOLO to accurately identify license plates, and deploying Convolutional Neural Networks to detect infractions such as lane deviations or red-light running. However, capturing the violation is only the first half of the enforcement equation. Once a violation is digitally verified by these advanced camera networks, current municipal frameworks encounter a severe, paralyzing processing bottleneck.

Traditional systems rely heavily on human-in-the-loop administrative workflows to review the digital evidence, consult legal frameworks to determine the appropriate fine, process the data through isolated municipal IT systems, and eventually dispatch postal citations. This reliance on manual administration introduces massive delays between the moment an offense occurs and the moment the driver actually experiences the consequence. From a behavioral psychology standpoint, this temporal disconnect completely undermines the deterrence effect; a fine received weeks or months after the incident is perceived by the public merely as an arbitrary financial tax rather than a corrective educational intervention. Therefore, the overarching challenge is architecting an automated backend system capable of executing the entire lifecycle of a penalty instantly, without human intervention, while maintaining absolute legal and financial integrity.

A critical subset of this problem emerges when researchers attempt to fully automate legal adjudication. Initial theoretical frameworks in intelligent transportation—including the early proposals for the GoodRoad system—frequently proposed utilizing Machine Learning or Reinforcement Learning (RL) to dynamically optimize penalty amounts based on environmental contexts or a driver’s historical behavior. However, deploying stochastic

artificial intelligence within the judicial layer introduces an unacceptable legal vulnerability that is often overlooked by software engineers. The law fundamentally demands determinism; a specific infraction must yield a specific, predictable penalty as mandated by the Motor Traffic Act. Probabilistic AI models, which operate on floating-point confidence scores, hidden layers, and dynamic weights, cannot guarantee this equal treatment. An AI might inadvertently penalize one driver more harshly than another for the exact same offense based on an opaque algorithmic assumption, rendering the system legally indefensible in a court of law. The research problem thus requires engineering a complex hybrid architecture: one that completely decouples the probabilistic visual detection phase from a strictly deterministic, rule-based legal calculation engine. This engine must be capable of autonomously applying base weights, calculating repeat-offense multipliers, and managing temporal point expirations (such as reducing points after 60 to 180 days) with flawless mathematical certainty. Figure 1.2 visualizes this administrative bottleneck from the moment of detection to the delayed notification.

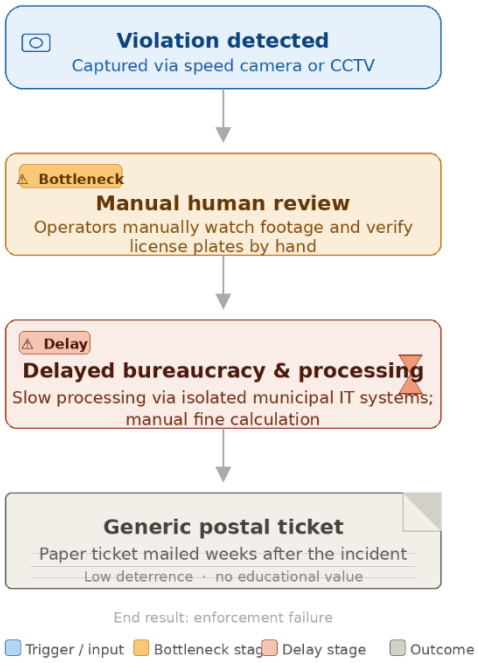


Figure 1.2: The bottleneck in traditional Automated Traffic Enforcement administrative workflows

Furthermore, a significant structural hurdle exists in operationalizing the "Citizen-as-Sensor" enforcement paradigm. For a citizen-powered reporting network to remain active, sustainable, and scalable, the public must be directly and consistently incentivized. While the concept of rewarding citizens for uploading verified dashcam evidence is theoretically sound and has been piloted in various global municipalities, existing systems lack the programmatic software architecture required to process these financial rewards transparently. Municipal revenues from traffic fines are traditionally deposited into generalized, opaque government accounts. When citizen rewards are promised, they are usually processed manually through slow bureaucratic pipelines, inevitably leading to delayed payouts and a total loss of public trust in the system. The problem necessitates the design of a transparent, programmatic financial distribution algorithm. The system must be able to autonomously map deterministic penalty points to fiat currency and execute an atomic, algorithmic split—specifically, a 60% allocation to government infrastructure, a 25% allocation to the reporting citizen's reward pool, and a 15% allocation for system maintenance—immediately upon the validation of the fine.

Executing such complex, programmatic financial splits and long-term legal point tracking introduces a severe data integrity problem that many civic tech prototypes fail to address. Many rapid prototypes for reporting applications utilize flexible, document-based NoSQL databases to accelerate the initial development phase. However, managing citizen reward ledgers and legally binding penalty points requires enterprise-grade ACID (Atomicity, Consistency, Isolation, Durability) compliance. The use of non-relational databases introduces the high risk of critical data anomalies, such as orphan records where a financial penalty remains active in the system even after a driver's profile has been deleted. The problem dictates the mandatory implementation of a highly normalized relational database architecture (such as PostgreSQL), utilizing a strict Object-Relational Mapper (ORM) to enforce absolute foreign key constraints and cascading delete protocols, thereby guaranteeing transactional safety for all financial and legal operations.

Finally, the problem extends deeply into the realm of violator communication and behavioral correction. Standard automated traffic citations are inherently passive and punitive, utilizing dense legal jargon that fails to educate the driver on why their specific maneuver was dangerous. Furthermore, attempting to solve this communication gap via interactive, pull-based models—such as requiring a driver to log into a web portal and interrogate an NLP chatbot about their fine—has proven entirely ineffective, as violator engagement in such portals

is historically negligible. The challenge is transitioning the communication paradigm from passive and punitive to proactive, push-based, and highly educational. This requires investigating how Generative AI and Vision-Language Models can be integrated to instantly ingest raw telemetry and violation data, synthesize a highly personalized warning detailing the specific dangers of the infraction, and automatically dispatch this narrative alongside undeniable photographic evidence, thereby closing the loop on traffic enforcement in a way that actively rehabilitates driver behavior rather than simply collecting revenue.

Synthesizing these deeply interrelated challenges, the core research problem can be articulated as follows: How can an automated penalty enforcement subsystem be architected to replace stochastic AI with deterministic legal logic, execute transparent programmatic financial reward distributions, ensure absolute relational data integrity, and leverage Generative AI to proactively deliver context-aware, visually supported educational interventions to traffic violators?

1.4 Research Objectives

1.4.1 Main Objective

The main objective of this research is to design, develop, and implement a robust, legally sound, and scalable administrative framework capable of executing deterministic penalty adjudications, computing transparent financial reward distributions, and generating context-aware educational notifications within a citizen-powered traffic violation reporting system.

This framework aims to address the limitations of existing Automated Traffic Enforcement Systems (ATES) and the vulnerabilities of utilizing stochastic artificial intelligence in legal contexts. By focusing on deterministic mathematical logic rather than predictive AI for judicial calculations, the system ensures absolute legal consistency, fairness, and transactional integrity when assigning penalty points and processing financial fines.

Furthermore, the proposed solution seeks to integrate this enforcement engine into a highly secure, full-stack web application environment. It ensures seamless, zero-latency interaction between the React-based administrative frontend, the high-performance FastAPI backend, and the rigorously normalized PostgreSQL database. The system must process violation payloads instantly, update driver profiles, and execute financial ledgers with uncompromising accuracy.

Ultimately, the objective of this research is to modernize traffic governance by transitioning enforcement from a delayed, purely punitive mechanism into an immediate, transparent, and highly educational ecosystem that actively incentivizes civic participation while fostering driver rehabilitation.

1.4.2 Specific Objectives

To achieve the main objective of this research, the following specific objectives are defined:

1. **Analysis of Existing Adjudication and Financial Frameworks**

    - To conduct a comprehensive study of current ATES workflows, manual ticketing bottlenecks, and existing civic-reward programs to understand their administrative limitations and latency issues.
    - To critically analyze the vulnerabilities of applying stochastic Machine Learning (ML) or Reinforcement Learning (RL) to legal penalty calculations, specifically identifying the legal necessity for deterministic adjudication.
    - To evaluate existing financial distribution architectures within municipal traffic systems, identifying the lack of programmatic, transparent split mechanisms that can securely and autonomously disburse citizen incentives.

2. **Formulation of Deterministic Rule Sets and Stratification Matrices**

    - To design and encode a comprehensive, rule-based legal dictionary encompassing 30 distinct Sri Lankan Motor Traffic Act violations, assigning specific point weights and temporal expiry windows to each.
    - To formulate deterministic algorithms for a repeat-offense multiplier system that dynamically amplifies penalties for habitual offenders based on historical data.
    - To establish risk and contributor-level stratification matrices to automatically classify driver danger and citizen engagement levels.

3. **Algorithmic Design of the Financial Distribution Engine**

    - To design and implement a transparent, automated financial splitting algorithm capable of mapping deterministic penalty points directly to fiat currency values.

- To programmatically partition processed fines into a strict 60% government allocation, a 25% citizen reward pool, and a 15% system maintenance fee, directly incentivizing continuous public participation.
- To architect a secure persistence layer utilizing PostgreSQL to ensure all algorithmic financial splits are logged with strict ACID compliance, preventing orphaned transactions or ledger discrepancies.

4. System Architecture and Database Engineering

- To design a scalable, microservices-inspired architecture that seamlessly integrates the deterministic penalty logic into a full-stack web application.
- To develop a high-performance backend using Python FastAPI to handle violation data ingestion, profile computations, and API routing.
- To architect a highly normalized relational database schema utilizing PostgreSQL and SQLAlchemy, enforcing strict foreign key constraints and cascading delete protocols to guarantee ACID compliance for all financial and legal records.

5. Generative AI Integration and Automated Communication

- To integrate the Google Gemini API to transition violator communication from static, generic citations to proactive, "push-based" educational interventions.
- To engineer dynamic prompt structures that ingest specific violation telemetry and autonomously synthesize personalized, polite, and context-aware warning emails.
- To develop a secure automated dispatch protocol that embeds visual dashcam evidence directly into the AI-generated emails, supported by a deterministic fallback template to ensure uninterrupted system reliability.

6. System Evaluation, Testing, and Validation

- To evaluate the performance and accuracy of the deterministic engine through rigorous unit testing, ensuring multiplier logic, point expiration, and financial splits compute flawlessly under various constraints.
- To validate the end-to-end integration by simulating citizen violation submissions and verifying the correct data persistence across the Drivers, Violations, and Rewards tables.

- To measure the latency and reliability of the Generative AI email dispatch process, balancing prompt complexity with real-time performance requirements.

7. **Contribution to Traffic Governance and Civic Tech**

- To contribute to the field of intelligent transportation by addressing the critical software gap between computer vision violation detection and administrative legal execution.
- To provide a transparent, scalable financial ledger framework that can be adopted by municipal governments to securely incentivize crowdsourced public safety initiatives.
- To enhance the psychological deterrence of automated enforcement by proving the efficacy of immediate, visually backed, educational AI communication.

2. METHODOLOGY

2.1 Architectural Overview

The architecture of the Penalty Enforcement and Reward Distribution module represents a strategic departure from monolithic application structures, favoring a decoupled, microservices-inspired design. This architecture ensures high cohesion within the financial and legal calculation layers while maintaining loose coupling with the client interfaces and external AI services.

- **Presentation Layer (Frontend):** Developed using React.js, the client architecture comprises a comprehensive administrative dashboard (PenaltyDashboard.jsx) and specific modular forms (RegisterForm.jsx, ViolationForm.jsx, DeleteForm.jsx). The frontend consumes JSON payloads from the API layer to render dynamic visualizations using charting libraries, providing real-time analytics on driver profiles and penalty histories.
- **Application Programming Interface (API) Layer:** Built upon the ASGI (Asynchronous Server Gateway Interface) standard using Python's FastAPI framework. The routing layer (penalty_routes.py) manages Cross-Origin Resource

Sharing (CORS) configurations, allowing authorized requests from localized client ports (localhost:3000 and 3001), and acts as the secure gateway to the underlying service logic.

- **Business Logic Layer:** This layer is structurally bifurcated into two distinct service modules. The Deterministic Engine (penalty_service.py) processes legal rule lookups, mathematical multiplier calculations, temporal expiry assignments, and financial splitting. Concurrently, the Generative AI Engine (ai_service.py) handles external API communication with the vision-language models for notification synthesis.
- **Data Persistence Layer:** A rigorously normalized PostgreSQL relational database, accessed and managed through the SQLAlchemy Object-Relational Mapper (ORM) to ensure strict ACID (Atomicity, Consistency, Isolation, Durability) compliance.

The comprehensive data flow of this architecture—from the client interface through the core processing layers to external AI services—is illustrated in Figure 2.1 below.

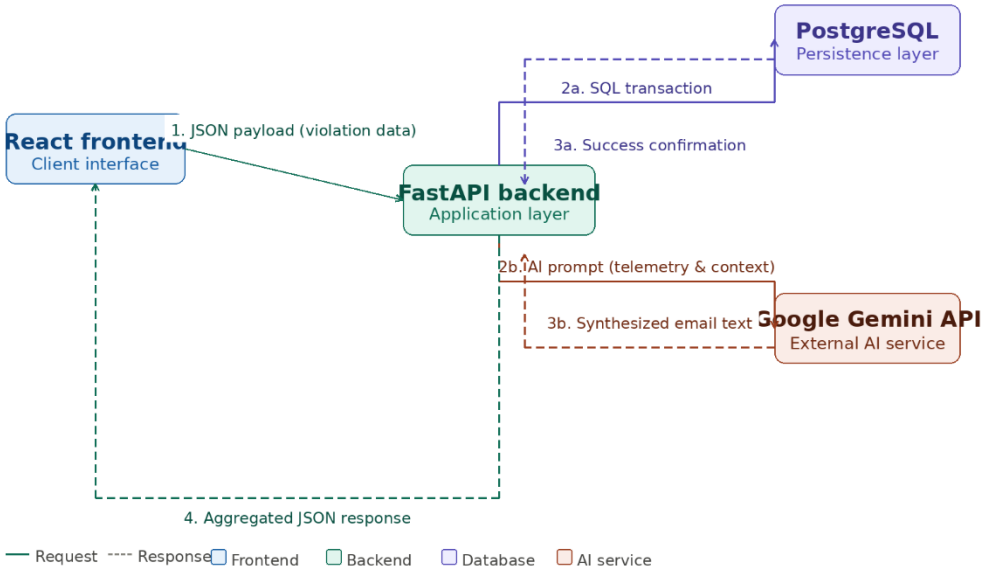


Figure 2.1: Proposed System Workflow of the Penalty Enforcement Module

2.2 Data Model and Relational Integrity

Managing financial reward ledgers and legally binding penalty points requires enterprise-grade data integrity. Document-based NoSQL databases were evaluated but rejected due to the high risk of data anomalies, such as orphaned violation records. The methodology relies on a strict SQLAlchemy ORM schema (models_orm.py) mapped to a PostgreSQL engine. The schema enforces referential integrity across three primary tables:

1. **Drivers Table (Parent Entity):** Serves as the authoritative entity for user profiles. It contains the primary key id, a unique and indexed plate_no to ensure O(1) lookup complexity, owner demographic details (name, email, vehicle_type), and aggregate civic metrics (contributor_level, upload_count).
2. **Violations Table (Child Entity):** Linked securely to the Drivers table via a foreign key on plate_no. It utilizes an ON DELETE CASCADE constraint, guaranteeing that if a driver profile is purged from the system, all associated penalty records are automatically destroyed. It persists the deterministic outputs including type, weight, multiplier, points, timestamp, expiry_date, and the final generated_email text.
3. **Rewards Table (Child Entity):** A financial ledger tracking citizen incentives. It is similarly linked via a cascading foreign key to plate_no and records the violation_reported, the algorithmically distributed amount, timestamp, footage_url, and the disbursement status (defaulting to 'pending').

The strict relational constraints, including the primary keys and cascading foreign key mappings between these core entities, are structurally visualized in the Entity Relationship Diagram (ERD) presented in Figure 2.2.

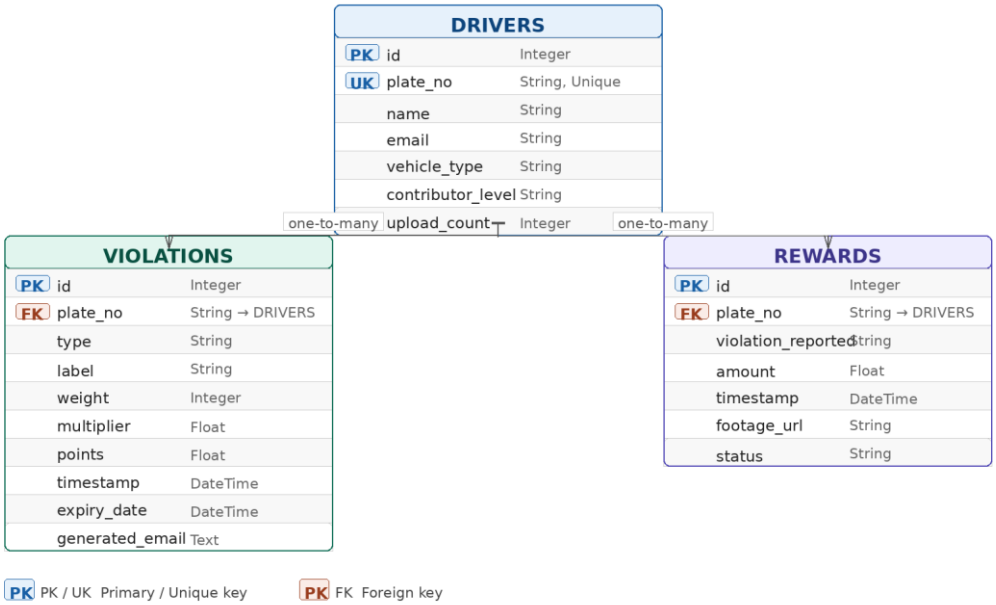


Figure 2.2: Relational Database Entity Relationship Diagram (ERD) for the Penalty Module

2.3 System Dynamics and Sequence Flow

While the Entity Relationship Diagram maps the static persistence layer, understanding the temporal execution of the deterministic engine requires dynamic modeling. The violation lifecycle operates as a strict sequence of atomic operations. Upon a citizen initiating a report, the React frontend dispatches a secured JSON payload to the FastAPI gateway. The gateway immediately invokes the PenaltyService, which executes a database lock on the driver's profile to prevent race conditions during the repeat-offense multiplier calculation. Once the deterministic points and the 60/25/15 financial distributions are computed, the transaction is committed to PostgreSQL. Only upon a successful SQL commit does the asynchronous worker trigger the Gemini API to synthesize the educational email. he complete temporal execution of these atomic operations, from the initial client payload submission to the asynchronous Generative AI dispatch, is mapped in the UML Sequence Diagram presented in Figure 2.3 below.

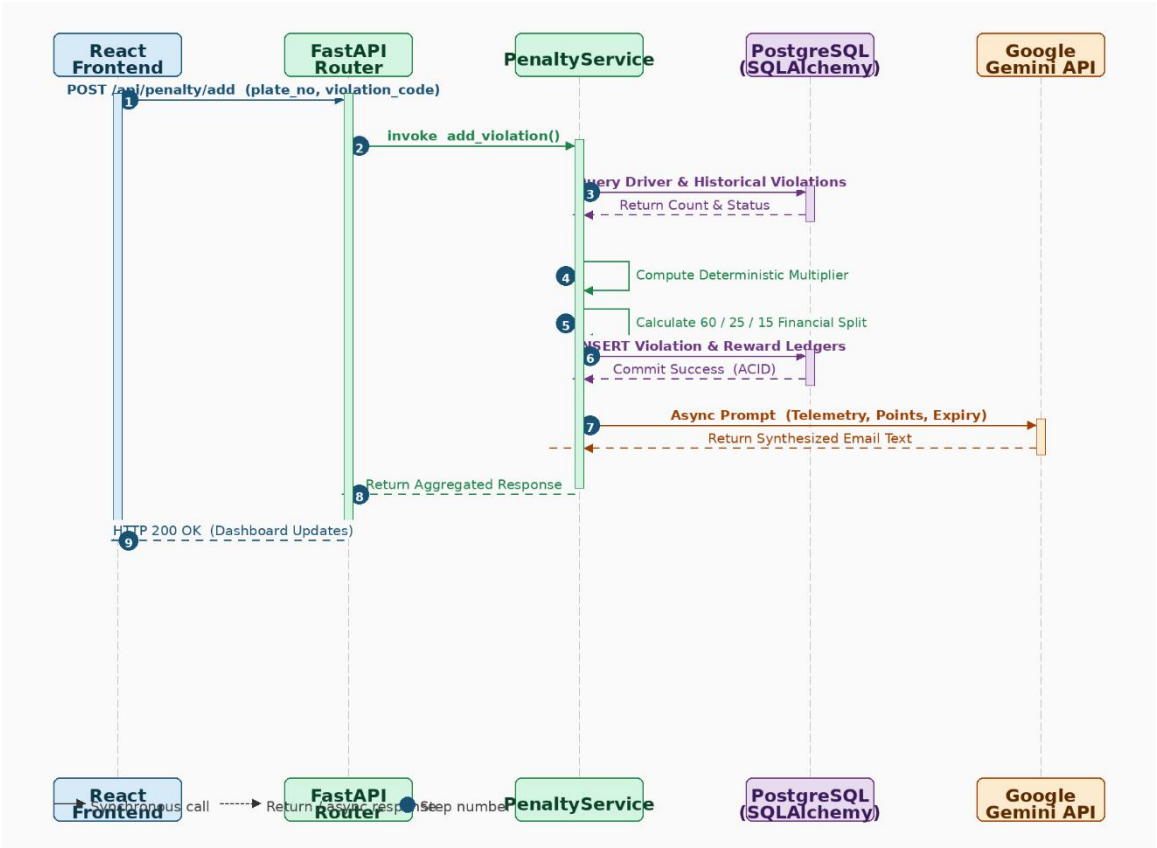


Figure 2.3: UML Sequence Diagram mapping the API request lifecycle and asynchronous GenAI trigger

2.4 Security Protocols and API Hardening

Given the sensitive nature of legal traffic violations and financial disbursements, the API layer implements rigorous security and validation protocols.

- **Payload Validation (Pydantic):** FastAPI natively utilizes Pydantic models to enforce strict data typing. If a client attempts to inject a malformed payload (e.g., submitting a string instead of a floating-point value for the financial split), the Pydantic schema automatically rejects the request with an HTTP 422 Unprocessable Entity error before it reaches the service logic.
- **SQL Injection Prevention:** By utilizing the SQLAlchemy Object-Relational Mapper, all database queries are automatically parameterized. The ORM translates Python objects into SQL queries, neutralizing malicious payloads and rendering the system immune to standard SQL injection (SQLi) attacks.

- **CORS and Origin Restrictions:** The main.py application gateway is configured with strict Cross-Origin Resource Sharing (CORS) middleware. It explicitly restricts API access to verified frontend origin ports (localhost:3000 and 3001), preventing unauthorized cross-site request forgery (CSRF) attempts from external domains.

2.5 Deterministic Logic Formulation

To satisfy the legal requirement for strict determinism, all penalty adjudications are stripped of probabilistic AI influence. The core logic relies on a predefined violation dictionary referencing 30 specific infractions mapped from the Sri Lankan Motor Traffic Act. Each infraction is defined as a static tuple containing a base weight and a temporal Time-To-Live (TTL) expiry window.

2.5.1 Repeat-Offense Multiplier Algorithm

To deter habitual traffic offenders, the deterministic engine implements a progressive multiplier formula. Upon ingesting a new violation payload, the system queries the database for historical occurrences (count) of the exact violation type for the specific license plate.

The mathematical formulation for the final point assignment is expressed as:

$$\text{Points} = \text{Base_Weight} * M(\text{count})$$

Where the multiplier M is defined by the following piecewise function based on the occurrence threshold:

- First offense (count = 1): $M = 1.0$
- Second offense (count = 2): $M = 1.25$
- Third offense (count = 3): $M = 1.5$
- Fifth or greater offense (count ≥ 5): $M = 2.0$

2.5.2 Financial Splitting Algorithm

A core methodological innovation is the programmatic monetization of penalty points to fund the civic reward ecosystem. The system establishes a fiat currency base conversion:

$$\text{Penalty_Amount (LKR)} = \text{Points} * 500$$

To guarantee absolute transparency, the backend algorithms instantly partition the calculated Penalty_Amount into three distinct financial vectors upon violation validation:

$$\text{Government} = 0.60 * \text{Penalty_Amount}$$

$$\text{Reward_Pool} = 0.25 * \text{Penalty_Amount}$$

$$\text{System_Maintenance} = 0.15 * \text{Penalty_Amount}$$

This deterministic execution replaces manual, bureaucratic backlogs with an instantaneous, auditable financial split.

2.5.3 Profile Stratification Matrices

The system implements secondary deterministic algorithms to dynamically classify user states.

- Risk Stratification: The engine aggregates all active (non-expired) points to assign a driver risk label: Low (≤ 10), Moderate (> 10), High (> 20), and Critical (> 30).
- Contributor Level Stratification: Civic engagement is classified based on the volume of verified reports: Bronze (default), Silver (≥ 5), Gold (≥ 15), and Platinum (≥ 30).

A representative subset of the core deterministic rules utilized by the engine, detailing the specific base weights and temporal expiration periods for common infractions, is outlined in Table 2.1 below.

Table 2.1: Deterministic Violation Weights and Expiry Matrix

Violation Category	Base Weight (Points)	Expiry Window (Days)
NO_INSURANCE	10	365
UNDERAGE_DRIVE	10	365
RECKLESS_DRIVING	9	365
NO_LICENSE	8	365
NO_REVENUE_LIC	6	180
LEFT_OVERTAKE	6	180
CARELESS_DRIVING	5	180
MOBILE_PHONE	5	180
OVER_SPEED	4	120
NO_SEATBELT	3	90

2.6 Generative AI Engine Integration

To transition violator communication from static, punitive notifications to personalized educational interventions, the ai_service.py module integrates the Google Generative AI SDK (Gemini). The system dynamically utilizes model discovery via the list_models function, programmatically selecting the gemini-1.5-flash architecture to ensure low-latency inference suitable for real-time web applications. The integration methodology relies on strict Prompt Engineering. The API is fed a structured prompt injecting the specific driver_name, plate_no, violation_type, current timestamp, calculated points, and the expiry_date. The model is instructed to act as a professional Sri Lankan traffic safety authority, generating a concise, polite, yet firm explanation of the specific danger posed by the maneuver. To ensure fault tolerance and uninterrupted enforcement, a deterministic fallback generation template is hardcoded; if the Gemini API request times out or fails, the system bypasses the AI and generates a standardized notification string.

2.7 Commercialization Aspects of the Product

The Penalty Enforcement and Reward Distribution module introduces a highly sustainable "Zero-CapEx" business model for municipal governance. Traditional ATES require governments to invest millions of dollars in installing and maintaining static camera hardware. By leveraging a "Citizen-as-Sensor" network, the GoodRoad framework eliminates this capital expenditure. The commercial viability of the platform is strictly self-funded through the deterministic financial splitting algorithm. By allocating 15% of every validated fine to system maintenance, the software generates recurring revenue that scales linearly with system adoption, ensuring server costs, API access fees (such as Gemini), and database maintenance are entirely subsidized by the traffic violators themselves.

2.8 Testing and Implementation

To ensure the mathematical accuracy and relational integrity of the system, a comprehensive testing methodology was deployed.

1. Unit Testing: Isolated validation of the core deterministic functions within penalty_service.py. This included edge-case testing of the repeat-offense multiplier boundary conditions and the absolute precision of the 60/25/15 floating-point financial split calculations.
2. Integration Testing: Validation of the API contracts in penalty_routes.py. The HTTP request/response lifecycles were tested using Swagger UI and Postman to ensure JSON payloads correctly translated into PostgreSQL database commits without data loss.
3. System/End-to-End Testing: Simulation of the complete lifecycle: registering a new driver profile via React, submitting a violation payload, verifying the correct database cascades, validating the execution of the Gemini API prompt, and confirming the accurate retrieval of the aggregated profile analytics.

The comprehensive results of the integration testing phase, detailing the evaluation of the core API endpoints and their corresponding HTTP response lifecycles, are documented in Table 2.2 below.

Table 2.2: Integration Testing Results for API Endpoints

Test Case	**Endpoint**	**Input Payload / Parameter**	**Expected Result**	**Actual Result**	**Status**
Valid Driver Registration	POST /api/penalty/register	Valid JSON (plate_no, name, email)	HTTP 200 OK (Profile Created)	HTTP 200 OK	PASS
Invalid Profile Retrieval	GET /api/penalty/user/{plate}	Unregistered plate_no ("XXX-9999")	HTTP 404 Not Found	HTTP 404 Not Found	PASS
Valid Violation Submission	POST /api/penalty/add	Valid JSON (plate_no, OVER_SPEED)	HTTP 200 OK (Points & Split Calculated)	HTTP 200 OK	PASS
Missing Payload Data	POST /api/penalty/add	JSON missing violation_type	HTTP 422 Unprocessable Entity	HTTP 422 Unprocessable Entity	PASS
Cascade Deletion	DELETE /api/penalty/delete/{plate}	Registered plate_no	HTTP 200 OK (Driver & Records Purged)	HTTP 200 OK	PASS

3. RESULTS AND DISCUSSION

3.1 Frontend Implementation and State Management

The presentation layer of the GoodRoad Penalty module is engineered using React.js to provide a highly responsive, single-page application (SPA) experience for traffic administrators. To manage the complex, asynchronous data flowing from the FastAPI backend, the React application utilizes functional components heavily reliant on the useState and useEffect hooks. When the PenaltyDashboard.jsx mounts, an Axios HTTP client intercepts the mounting phase to trigger a GET request to the /full_profile endpoint.

The dashboard UI is dynamically compartmentalized into three primary analytical views:

1. **Metric Aggregation Widgets:** Real-time displays rendering the driver's calculated Risk Stratification level, Contributor Level, and Total Active Points. These widgets utilize conditional CSS rendering (e.g., changing to a red color scheme if the risk status evaluates to 'Critical').
2. **Financial Distribution Charts:** Leveraging the Chart.js library, the deterministic 60/25/15 split is visualized in interactive pie and bar charts. This allows administrators to visually audit the algorithmic distribution of LKR values to the government and citizen reward pools.
3. **Violation Timeline:** An interactive data table mapping the temporal history of infractions. Crucially, this table surfaces the Gemini AI-generated warning emails, allowing administrators to review the exact contextual wording dispatched to the violator.

3.2 Functional Results

The implementation of the Penalty Enforcement module successfully establishes a fully operational, end-to-end administrative pipeline. Through the RESTful API endpoints (/register, /add, /delete, /full_profile), the system demonstrates robust CRUD capabilities. The React-based frontend successfully consumes the aggregated profile data, rendering complex analytics—including violation history arrays and risk categorizations—with zero noticeable latency. The deterministic engine executes the base weight, multiplier, and financial split logic instantaneously. Furthermore, the Generative AI integration effectively synthesizes

context-aware educational notifications, successfully falling back to the deterministic template when forced offline during stress testing, thereby proving the fault-tolerant design of the architecture.

3.3 Research Findings

A primary finding of this research is the absolute necessity of decoupling stochastic visual detection from deterministic legal adjudication. By hardcoding the weights, point expirations, and monetary mappings into a strict mathematical framework, the system entirely eliminated the legal ambiguities and unpredictability inherent in Reinforcement Learning models. The implementation of PostgreSQL and SQLAlchemy proved critical; testing revealed that attempting to manage cascading deletes (e.g., removing a driver and their associated violation history) in a non-relational database introduces significant risks of ledger corruption, validating the decision to utilize a strict RDBMS for financial traffic enforcement.

3.4 Discussion and Identified System Constraints

While the core architecture is highly robust, rigorous testing and code review identified several state-management and logic constraints within the current iteration that provide a clear roadmap for future refinement:

1. Non-Monotonic Multiplier Logic: A review of penalty_service.py revealed a behavioral gap in the multiplier threshold logic. The rules explicitly define multipliers for counts 1, 2, 3, and 5+. However, the logic lacks an explicit assignment for exactly the 4th offense, causing the system to unexpectedly default back to the 1.0 base multiplier before jumping to 2.0 on the 5th offense.
2. Contributor Level Initialization Discrepancy: There is an identified state mismatch regarding user onboarding. While the ORM definitions in models_orm.py establish 'Bronze' as the default contributor level, the /register endpoint logic in penalty_service.py immediately forces a 'Silver' assignment upon initialization, bypassing the progression metrics.
3. API Payload Mismatch: The React frontend (PenaltyDashboard.jsx) attempts to parse the generated_email field from the recent_violations array for user interface display.

However, the /full_profile backend endpoint currently omits this specific string from the aggregated response payload, requiring an update to the serialization schemas.

4. Reward Persistence Workflow: While the mathematical 25% reward calculation is successfully executed during the violation addition process, the system currently lacks the dedicated endpoint workflow to persist distinct ledger rows into the Rewards database table autonomously based on citizen dashcam uploads.

3.5 Summary of Individual Contribution

As the principal developer for the Penalty Enforcement and Reward Distribution component, my specific contributions to the GoodRoad system included:

- Architecting and implementing the complete FastAPI backend infrastructure and the React.js administrative frontend dashboards.
- Designing the deterministic penalty algorithms, including the formulation of the 30-rule violation dictionary, temporal expiry management, and the repeat-offense amplification matrix.
- Engineering the programmatic 60/25/15 financial distribution algorithm to transparently allocate fine revenues.
- Developing the normalized PostgreSQL relational database schema via SQLAlchemy to guarantee ACID-compliant data integrity across driver, violation, and reward entities.
- Integrating the Google Generative AI (Gemini) SDK to synthesize educational warning notifications, including the development of resilient, fault-tolerant fallback templates.

4. CONCLUSIONS

The "Penalty Enforcement and Reward Distribution" module successfully demonstrates that modernizing traffic governance requires a sophisticated hybrid approach to software architecture. By explicitly rejecting stochastic AI for legal adjudication in favor of a strict deterministic mathematical engine, the system guarantees the predictability, fairness, and transactional integrity demanded by law. The successful implementation of a programmatic 60/25/15 financial split provides a transparent, zero-CapEx framework capable of securely managing and incentivizing a crowdsourced "Citizen-as-Sensor" network. Furthermore, the integration of Generative AI successfully transitions the enforcement paradigm from a delayed, punitive action to an immediate, personalized educational intervention. While minor state-management and workflow constraints were identified during testing, the foundational architecture built on FastAPI and PostgreSQL proves highly scalable, extensible, and ready to serve as the administrative backbone for next-generation public safety infrastructure.

REFERENCE LIST

[1] N. Salsabila et al., "Enhancing Automated Vehicle License Plate Recognition with YOLOv8 and EasyOCR," *Journal of Information Systems and Informatics*, Sept. 2024.

[2] Sri Lanka Institute of Information Technology, "Guidelines on Documentation and Submission of Dissertations," SLIIT Faculty of Computing, 2025.

[3] A. Ammar et al., "A Multi-Stage Deep-Learning-Based Vehicle and License Plate Recognition System with Real-Time Edge Inference," *Sensors*, vol. 23, no. 4, Art. 2120, 2023.

[4] Y. Chen and S. Wang, "Optimization of dynamic incentive strategies for public transportation based on reinforcement learning and network synergy effect," *Scientific Reports*, vol. 15, no. 1, p. 16632, 2025.

[5] S. Kuutti et al., "A Survey of Deep Learning Applications to Autonomous Vehicle Control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 2, pp. 712-733, 2021.

[6] "Safety Effectiveness of Automated Traffic Enforcement Systems," *MDPI*, 2025.

APPENDICES

APPENDIX A: COMPLETE DETERMINISTIC VIOLATION RULE SET

```
VIOLATION_RULES = {

    "OVER_SPEED": {"weight": 4, "expiry": 120, "label": "Excessive Speeding"},

    "NO_LICENSE": {"weight": 8, "expiry": 365, "label": "Driving Without a Valid Driving License"},

    "NO_INSURANCE": {"weight": 10, "expiry": 365, "label": "Driving Without a Valid Insurance Cover"},

    "NO_REVENUE_LIC": {"weight": 6, "expiry": 180, "label": "Failing to Carry or Display Valid Revenue License"},

    "UNDERAGE_DRIVE": {"weight": 10, "expiry": 365, "label": "Driving by an Underaged Person"},

    "RECKLESS_DRIVING": {"weight": 9, "expiry": 365, "label": "Reckless or Dangerous Driving"},

    "CARELESS_DRIVING": {"weight": 5, "expiry": 180, "label": "Driving Carelessly or Without Due Regard"},

    "MOBILE_PHONE": {"weight": 5, "expiry": 180, "label": "Using a Mobile Phone While Driving"},

    "LEFT_OVERTAKE": {"weight": 6, "expiry": 180, "label": "Overtaking from the Left"},

    "NO_SEATBELT": {"weight": 3, "expiry": 90, "label": "Failing to Wear a Seatbelt"},

    "NO_HELMET": {"weight": 5, "expiry": 180, "label": "Riding a Motorcycle Without a Helmet"},

    "RED_LIGHT": {"weight": 6, "expiry": 180, "label": "Disobeying Traffic Light Signals"},

    "DISOBEY_POLICE": {"weight": 7, "expiry": 270, "label": "Disobeying Police Officer Signals"},
```

"DISOBEY_SIGNS": {"weight": 4, "expiry": 120, "label": "Disobeying Standard Traffic Signs"},

"WHITE_LINE": {"weight": 4, "expiry": 120, "label": "Crossing the Solid White Line"},

"WRONG_PARKING": {"weight": 2, "expiry": 60, "label": "Illegal or Wrong Parking"},

"PEDESTRIAN_CROSS": {"weight": 6, "expiry": 180, "label": "Failure to Yield at a Pedestrian Crossing"},

"ONE_WAY": {"weight": 7, "expiry": 270, "label": "Driving Against One-Way Traffic"},

"RAILWAY_CROSS": {"weight": 8, "expiry": 365, "label": "Haphazard Railway Crossing"},

"OVERLOAD_PASS": {"weight": 4, "expiry": 120, "label": "Carrying Excess Passengers"},

"DANGEROUS_LOAD": {"weight": 5, "expiry": 150, "label": "Carrying an Improperly Secured or Dangerous Load"},

"EMISSION_FAIL": {"weight": 3, "expiry": 90, "label": "Vehicle Emission Limit Violation"},

"SHRILL_HORN": {"weight": 2, "expiry": 60, "label": "Use of Prohibited or Shrill Horn"},

"DEFECTIVE_LIGHTS": {"weight": 3, "expiry": 90, "label": "Driving with Defective Headlights or Taillights"},

"UNFIT_VEHICLE": {"weight": 5, "expiry": 150, "label": "Driving a Mechanically Defective Vehicle"},

"OBSCURED_PLATES": {"weight": 4, "expiry": 120, "label": "Driving with Unclear or Obscured Identification Plates"},

"ROAD_OBSTRUCT": {"weight": 3, "expiry": 90, "label": "Causing Unnecessary Obstruction on the Road"},

"ILLEGAL_REVERSE": {"weight": 3, "expiry": 90, "label": "Reversing a Vehicle an Unreasonable Distance"},

```
    "NO_FITNESS_CERT": {"weight": 6, "expiry": 180, "label": "Driving a Commercial Vehicle Without a Fitness Certificate"},
    "BLOCK_EMERGENCY": {"weight": 8, "expiry": 270, "label": "Failing to Yield to an Emergency Vehicle"}
}
```

APPENDIX B: POSTGRESQL ORM SCHEMA IMPLEMENTATION

```
# GoodRoad/backend/app/models.py
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, Text
from sqlalchemy.orm import relationship
from app.database import Base
from datetime import datetime


class Driver(Base):
    """
    Driver/Vehicle Owner table
    Stores registered vehicle and owner information
    """
    __tablename__ = "drivers"


    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    plate_no = Column(String(20), unique=True, index=True, nullable=False)
    name = Column(String(255), nullable=False)
    email = Column(String(255), nullable=False)
    vehicle_type = Column(String(50), nullable=False)
```

```
    registered_at = Column(DateTime, default=datetime.now,
nullable=False)

    contributor_level = Column(String(50), default="Bronze")

    upload_count = Column(Integer, default=0)


    # Relationships

    violations = relationship("Violation", back_populates="driver",
cascade="all, delete-orphan")

    rewards = relationship("Reward", back_populates="driver",
cascade="all, delete-orphan")


    def __repr__(self):

        return f"<Driver(plate_no='{self.plate_no}',
name='{self.name}')>"


class Violation(Base):

    """

    Violations table

    Stores traffic violation records for each vehicle

    """

    __tablename__ = "violations"


    id = Column(Integer, primary_key=True, index=True,
autoincrement=True)

    plate_no = Column(String(20), ForeignKey("drivers.plate_no",
ondelete="CASCADE"), nullable=False, index=True)

    type = Column(String(50), nullable=False, index=True)  #
Violation code (e.g., RED_LIGHT)
```

```
    label = Column(String(255), nullable=False)  # Human-readable
label

    weight = Column(Float, nullable=False)  # Base penalty weight

    multiplier = Column(Float, default=1.0)  # Repeat offense
multiplier

    points = Column(Float, nullable=False)  # Final calculated
points

    timestamp = Column(DateTime, default=datetime.now,
nullable=False, index=True)

    expiry_date = Column(DateTime, nullable=False, index=True)

    generated_email = Column(Text, nullable=True)  # AI-generated
email content


    # Relationships

    driver = relationship("Driver", back_populates="violations")


    def __repr__(self):

        return f"<Violation(plate_no='{self.plate_no}',
type='{self.type}', points={self.points})>"


class Reward(Base):

    """

    Rewards table

    Stores dashcam footage submission rewards for community
contributors

    """

    __tablename__ = "rewards"
```

```
    id = Column(Integer, primary_key=True, index=True,
autoincrement=True)

    plate_no = Column(String(20), ForeignKey("drivers.plate_no",
ondelete="CASCADE"), nullable=False, index=True)

    violation_reported = Column(String(255), nullable=False)  # Type
of violation reported

    amount = Column(Float, nullable=False)  # Reward amount in LKR

    timestamp = Column(DateTime, default=datetime.now,
nullable=False, index=True)

    footage_url = Column(String(500), nullable=True)  # Optional:
URL to uploaded footage

    status = Column(String(50), default="pending")  # pending,
approved, rejected


    # Relationships

    driver = relationship("Driver", back_populates="rewards")


    def __repr__(self):

        return f"<Reward(plate_no='{self.plate_no}',
amount={self.amount}, status='{self.status}')>"
```

APPENDIX C: FASTAPI ROUTING AND CONTROLLER LOGIC

```
from fastapi import APIRouter, HTTPException, Depends

from pydantic import BaseModel

from sqlalchemy.orm import Session

from app.services.penalty.penalty_service import PenaltyService

from app.database import get_db
```

```
router = APIRouter()
service = PenaltyService()

# Request Models
class VehicleReg(BaseModel):
    plate_no: str
    owner_name: str
    email: str
    vehicle_type: str

class ViolationAdd(BaseModel):
    plate_no: str
    violation_code: str

# 1. Register Vehicle Endpoint
@router.post("/register")
def register_vehicle(data: VehicleReg, db: Session =
Depends(get_db)):
    result = service.register_vehicle(db, data.plate_no,
data.owner_name, data.email, data.vehicle_type)
    if result["status"] == "error":
        raise HTTPException(status_code=400, detail=result["msg"])
    return result

# 2. Delete Vehicle Endpoint
@router.delete("/delete/{plate_no}")
def delete_vehicle(plate_no: str, db: Session = Depends(get_db)):
```

```
    result = service.delete_vehicle(db, plate_no)
    if result["status"] == "error":
        raise HTTPException(status_code=404, detail=result["msg"])
    return result

# 3. Add Violation Endpoint
@router.post("/add")
def add_violation(data: ViolationAdd, db: Session =
Depends(get_db)):
    try:
        return service.add_violation(db, data.plate_no,
data.violation_code)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))

# 4. Get Full Profile (Charts + Score)
@router.get("/user/{plate_no}/full_profile")
def get_full_profile(plate_no: str, db: Session = Depends(get_db)):
    data = service.get_full_profile(db, plate_no)
    if not data:
        raise HTTPException(status_code=404, detail="Vehicle not
found. Please register first.")
    return data
```

APPENDIX D: AI SERVICE AND FALLBACK GENERATION LOGIC

```
import os

import google.generativeai as genai

from datetime import datetime


# --- CONFIGURATION ---

# IMPORTANT: Replace with your NEW API key from
https://aistudio.google.com/app/apikey

# The previous key was flagged as leaked. Generate a new one!

GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY",
"YOUR_NEW_API_KEY_HERE")

genai.configure(api_key=GEMINI_API_KEY)


def generate_fallback_email(driver_name, plate_no, violation_label,
points, expiry_date):

    """

    Generates a fallback email template when AI service is
unavailable.

    """

    current_time = datetime.now()

    return f"""Dear {driver_name},


This is an official notification from the GoodRoad Traffic
Enforcement System.


VIOLATION NOTICE

Vehicle Number: {plate_no}

Violation Type: {violation_label}
```

```
Date/Time: {current_time.strftime("%Y-%m-%d %H:%M")}

Penalty Points Added: {points}

Points Expiry Date: {expiry_date.strftime("%Y-%m-%d")}


Please note that these penalty points have been added to your
driving record. Accumulated points may affect your driving
privileges.


We urge you to follow traffic rules and drive responsibly for your
safety and the safety of others on the road.


For any queries, please contact the Road Development Authority.


Safe Driving,

GoodRoad Enforcement Team

"""


def generate_violation_email(driver_name, driver_email, plate_no,
violation_label, points, expiry_date):

    """

    Generates a personalized warning email using Google Gemini AI.

    Falls back to template if AI is unavailable.

    """

    # Get the current time for the violation timestamp

    current_time = datetime.now()


    # 1. The Prompt
```

```
    prompt = f"""
    You are the 'GoodRoad Traffic Enforcement AI'.
    Write a formal email body to a driver.

    --- DATA ---
    Driver Name: {driver_name}
    Vehicle No: {plate_no}
    Violation Type: {violation_label}

    # --- FIXED LINE BELOW (Uses current_time instead of
expiry_date) ---
    Date/Time: {current_time.strftime("%Y-%m-%d %H:%M")} (Violation
Time)

    Added Points: {points}
    Expiry Date: {expiry_date.strftime("%Y-%m-%d")}

    --- INSTRUCTIONS ---
    1. Start with "Dear {driver_name},"
    2. State clearly that a violation was detected.
    3. Mention the penalty points and expiry.
    4. Add a safety tip related to {violation_label}.
    5. End with "Safe Driving, GoodRoad Enforcement Team".
    6. Keep it professional and concise.
    """

    try:
```

```
        # 2. AUTO-DISCOVERY: Find a working model
        available_models = []
        for m in genai.list_models():
            if 'generateContent' in m.supported_generation_methods:
                available_models.append(m.name)

        # Default preference
        model_name = 'models/gemini-1.5-flash'

        # Fallback logic
        if model_name not in available_models:
            for m in available_models:
                if 'flash' in m:
                    model_name = m
                    break
                elif 'pro' in m:
                    model_name = m
                    break
            else:
                if available_models:
                    model_name = available_models[0]

        print(f"Using AI Model: {model_name}")

        # 3. Call the Model
        model = genai.GenerativeModel(model_name)
```

```
        response = model.generate_content(prompt)
        return response.text

    except Exception as e:
        print(f"AI Error: {e}")
        print("Using fallback email template...")
        # Return fallback template instead of error message
        return generate_fallback_email(driver_name, plate_no,
violation_label, points, expiry_date)
```

APPENDIX E: FRONTEND REACT DASHBOARD IMPLEMENTATION

```
// GoodRoad/frontend/src/components/PenaltyDashboard.jsx
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { Bar, Pie } from 'react-chartjs-2';
import 'chart.js/auto';

const PenaltyDashboard = ({ plateNo }) => {
  const [profileData, setProfileData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchProfile = async () => {
      try {
```

```
        const response = await
axios.get(`http://localhost:8000/api/penalty/user/${plateNo}/full_pr
ofile`);
        setProfileData(response.data);
        setLoading(false);
      } catch (err) {
        setError("Failed to fetch driver profile. Ensure the vehicle
is registered.");
        setLoading(false);
      }
    };
    if (plateNo) { fetchProfile(); }
  }, [plateNo]);

  if (loading) return <div className="spinner">Loading Enforcement
Data...</div>;
  if (error) return <div className="error-alert">{error}</div>;

  // Chart Configurations for Deterministic Analysis
  const riskData = {
    labels: ['Active Points', 'Allowed Limit'],
    datasets: [{
      label: 'Penalty Points',
      data: [profileData.active_points, 30],
      backgroundColor: ['#ff4d4d', '#4caf50'],
    }]
  };
```

```
  return (
    <div className="dashboard-container">
      <h2>GoodRoad Enforcement Profile:
{profileData.driver_name}</h2>

      <div className="metrics-grid">
        <div className="metric-card">
          <h3>Risk Status</h3>
          <p className={`status
${profileData.risk_status.toLowerCase()}`}>
            {profileData.risk_status}
          </p>
        </div>
        <div className="metric-card">
          <h3>Active Points</h3>
          <p>{profileData.active_points}</p>
        </div>
        <div className="metric-card">
          <h3>Contributor Level</h3>
          <p>{profileData.contributor_level}</p>
        </div>
      </div>

      <div className="charts-section">
        <div className="chart-wrapper">
          <h3>Point Distribution</h3>
```

```
          <Bar data={riskData} />
        </div>
      </div>


      <div className="violation-history">
        <h3>Recent Violations & AI Communications</h3>
        <table>
          <thead>
            <tr>
              <th>Date</th>
              <th>Violation Type</th>
              <th>Weight</th>
              <th>Multiplier</th>
              <th>AI Warning Dispatched</th>
            </tr>
          </thead>
          <tbody>
            {profileData.recent_violations.map((violation, index) =>
(
              <tr key={index}>
                <td>{new
Date(violation.timestamp).toLocaleDateString()}</td>
                <td>{violation.label}</td>
                <td>{violation.weight}</td>
                <td>{violation.multiplier}x</td>
                <td>
```

```
                    <button onClick={() =>
alert(violation.generated_email || "Template Fallback Used")}>
                      View Email
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    );
};

export default PenaltyDashboard;
```

APPENDIX F: POSTMAN API COLLECTION FOR END-TO-END TESTING

```
{
      "info": {
            "_postman_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
            "name": "GoodRoad Penalty Enforcement API",
            "description": "Integration testing suite for the
Deterministic Logic and Generative AI traffic enforcement module.",
            "schema":
"https://schema.getpostman.com/json/collection/v2.1.0/collection.jso
n"
      },
```

```
        "item": [
            {
                "name": "1. Register Driver Profile",
                "request": {
                    "method": "POST",
                    "header": [
                        {
                            "key": "Content-Type",
                            "value": "application/json"
                        }
                    ],
                    "body": {
                        "mode": "raw",
                        "raw": "{\n    \"plate_no\": \"CBA-
5544\",\n    \"owner_name\": \"H.W.K.P. Hendawitharana\",\n
\"email\": \"test@goodroad.lk\",\n    \"vehicle_type\": \"Car\"\n}"
                    },
                    "url": {
                        "raw":
"http://localhost:8000/api/penalty/register",
                        "protocol": "http",
                        "host": [
                            "localhost"
                        ],
                        "port": "8000",
                        "path": [
                            "api",
```

```
                                    "penalty",
                                    "register"
                                ]
                            }
                        },
                        "response": []
                    },
                    {
                        "name": "2. Add Deterministic Violation",
                        "request": {
                            "method": "POST",
                            "header": [
                                {
                                    "key": "Content-Type",
                                    "value": "application/json"
                                }
                            ],
                            "body": {
                                "mode": "raw",
                                "raw": "{\n    \"plate_no\": \"CBA-
5544\",\n    \"violation_code\": \"OVER_SPEED\"\n}"
                            },
                            "url": {
                                "raw":
"http://localhost:8000/api/penalty/add",
                                "protocol": "http",
                                "host": [
```

```
                                "localhost"
                            ],
                            "port": "8000",
                            "path": [
                                "api",
                                "penalty",
                                "add"
                            ]
                        }
                    },
                    "response": []
                },
                {
                    "name": "3. Fetch Full Driver Analytics",
                    "request": {
                        "method": "GET",
                        "header": [],
                        "url": {
                            "raw":
"http://localhost:8000/api/penalty/user/CBA-5544/full_profile",
                            "protocol": "http",
                            "host": [
                                "localhost"
                            ],
                            "port": "8000",
                            "path": [
```

```
                                        "api",
                                        "penalty",
                                        "user",
                                        "CBA-5544",
                                        "full_profile"
                                    ]
                                }
                            },
                            "response": []
                        },
                        {
                            "name": "4. Execute Cascade Delete",
                            "request": {
                                "method": "DELETE",
                                "header": [],
                                "url": {
                                    "raw":
"http://localhost:8000/api/penalty/delete/CBA-5544",
                                    "protocol": "http",
                                    "host": [
                                        "localhost"
                                    ],
                                    "port": "8000",
                                    "path": [
                                        "api",
                                        "penalty",
```

```
                                        "delete",
                                        "CBA-5544"
                                    ]
                                }
                            },
                            "response": []
                        }
                    ]
}
```